Instituto Profesional Santo tomas

Ingeniería en Informática

HIRATA

Fleet Management System

Informe Técnico

Etapa 1 — Avance del Proyecto

Asignatura:	Integración de Competencias I
Docente:	Jose Fuentes Verdugo
Integrantes:	Benjamín Alvarez Patricio López Víctor LLanos Duvan Alfonso Cuta Eduardo Fernández
Fecha:	20 de Abril 2026
Versión:	1.0 — Etapa 1

Valdivia, Chile | 2026

Índice

1 Introducción **3**

1.1 Antecedentes del Proyecto 3

1.2 Objetivos 3

1.2.1 Objetivo General 3

1.2.2 Objetivos Específicos 3

1.3 Alcance 4

1.4 Tecnologías Utilizadas 4

2 Desarrollo **5**

2.1 Arquitectura del Sistema 5

2.1.1 Diagrama de Arquitectura General 5

2.1.2 Estructura del Proyecto 5

2.2 Base de Datos — Diseño Relacional 6

2.2.1 Modelo de Entidades 6

2.2.2 Diagrama Entidad-Relación Simplificado 7

2.2.3 Aspectos Técnicos Destacados del Esquema 7

2.3 Módulo de Autenticación y Sesiones 7

2.4 RF-03—Notificación de Mantenimiento 8

2.4.1 Funcionamiento del Módulo 9

2.4.2 Flujo de Asignación de Camiones 10

2.5 RF-04 — Gestión de Registros de Mantenimiento 11

2.5.1 Ciclo de Vida de un Mantenimiento 11

2.5.2 Lógica de Transición de Estado del Camión 11

2.5.3 Operaciones CRUD del Módulo 12

2.6 RF-05 — Pruebas Unitarias Automatizadas 12

2.6.1 Stack de Pruebas 12
2.6.2 Suites de Pruebas Definidas 13
2.6.3 Flujo de Ejecución 13
2.7 Sistema de Notificaciones 13

3 Conclusión **15**
3.1 Resumen de Logros 15
3.2 Aspectos Técnicos Destacados 15
3.3 Trabajo Pendiente para Etapas Siguientes 16
3.4 Reflexión Final 16

4 Documentación del Proyecto **17**
4.1 Descripción del Proyecto 17
4.2 Arquitectura del Sistema 17
4.2.1 Frontend 17
4.2.2 Backend 18
4.2.3 Base de Datos (MySQL) 18
4.3 Instalación y Despliegue 18
4.3.1 Requisitos Previos 18
4.3.2 Configuración del Servidor 18
4.3.3 Acceso al Cliente 19

1. Introducción

El presente informe corresponde a la **Primera Etapa** del proyecto desarrollado en el marco de la asignatura *Integración de Competencias (IEI-091)*, cuyo objetivo es diseñar e implementar una solución informática integral para la empresa de transportes **Hirata**.

1.1 Antecedentes del Proyecto

Transportes Hirata opera una flota de camiones que requiere un control riguroso de asignaciones de conductores, seguimiento de kilometraje y gestión de mantenimiento preventivo. Antes de la implementación de este sistema, estos procesos se realizaban de forma manual, lo que generaba ineficiencias, riesgos de omisión de servicios técnicos y dificultades en la trazabilidad de la flota.

El sistema de información desarrollado automatiza el ciclo de vida completo de la flota, centralizando la información en una base de datos relacional y exponiendo una API REST segura que sirve tanto a administradores como a conductores.

1.2 Objetivos

1.2.1 Objetivo General

Desarrollar un sistema web de gestión de flota que permita a Transportes Hirata administrar de manera eficiente las asignaciones de vehículos, el registro de kilometraje y el control del mantenimiento preventivo.

1.2.2 Objetivos Específicos

- Implementar un módulo de autenticación seguro con gestión de sesiones mediante JWT y cookies.
- Diseñar y construir un esquema de base de datos relacional (MySQL) que soporte todos los procesos operativos del sistema.

- Desarrollar una API REST (Node.js + Express) con arquitectura por capas (DAO / Service / Controller).
- Implementar la lógica de asignación y reasignación de camiones con validaciones transaccionales.
- Implementar el módulo de registro de kilometraje con detección automática de necesidad de mantenimiento.
- Desarrollar el módulo de gestión de mantenimiento con ciclo completo de estados.
- Crear un sistema de notificaciones automáticas para alertar a los roles correspondientes.
- Diseñar pruebas unitarias automatizadas con Jest y Supertest para garantizar la estabilidad del sistema.

1.3 Alcance

Esta primera etapa comprende la definición de requerimientos funcionales (RF-03, RF-04 y RF-05), el diseño e implementación del backend completo (API REST, base de datos, lógica de negocio) y la especificación de las pruebas que validan el sistema. La capa de presentación (frontend) constituye un componente complementario que interactúa con la API definida.

1.4 Tecnologías Utilizadas

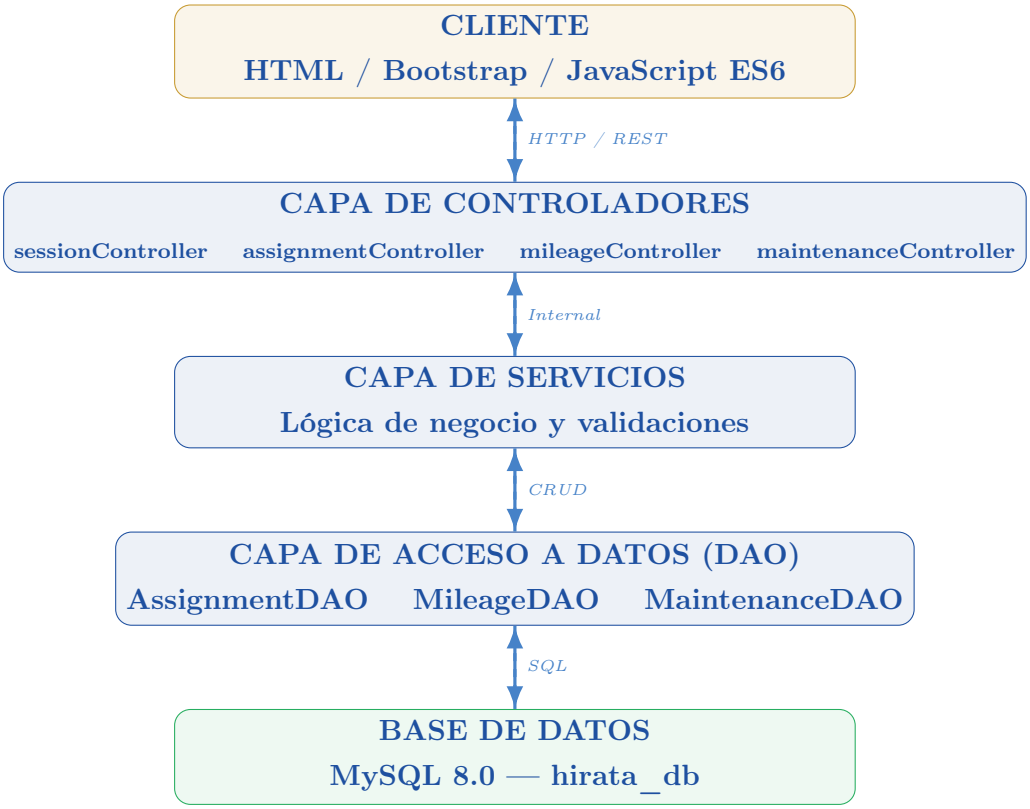
Tecnología	Uso en el Proyecto
Node.js + Express	Servidor de API REST, manejo de rutas y middleware
MySQL 8.0	Base de datos relacional, transacciones ACID
JWT + Bcrypt	Autenticación segura, gestión de sesiones y tokens
Jest + Supertest	Framework de pruebas unitarias automatizadas
HTML + Bootstrap	Interfaces de usuario del cliente web
JavaScript ES6+	Lógica de cliente (módulos, async/await)

2. Desarrollo

2.1 Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas en el backend, separando claramente las responsabilidades de cada componente.

2.1.1 Diagrama de Arquitectura General



2.1.2 Estructura del Proyecto

El proyecto se organiza en dos grandes carpetas:

Estructura de Directorios

```
hirata-fleet-api/
  client/                    # Frontend
    login.html
    admflota.html
    js/
```

```
      app.js                # Sesiones globales y proteccion de rutas
      login.js
      admflota.js
      driver-mileage.js
  server/                    # Backend (API REST)
    src/
      controllers/           # Reciben peticiones HTTP
      services/              # Logica de negocio
      dao/                   # Acceso a base de datos
        models/              # Definicion de tablas
      utils/                 # dbConnection, hash, etc.
      dto/                   # Objetos de transferencia
      routes/                # Definicion de endpoints
    tests/                   # Pruebas unitarias (Jest)
    index.js                 # Punto de entrada del servidor
```

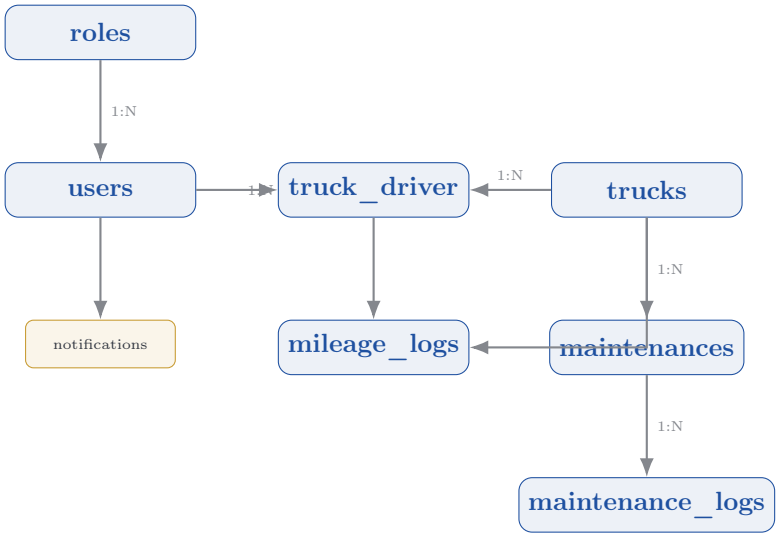
2.2 Base de Datos — Diseño Relacional

2.2.1 Modelo de Entidades

La base de datos `hirata_db` está diseñada sobre MySQL 8.0 y consta de las siguientes tablas principales:

Tabla	Clave Principal	Descripción
`roles`	`id`	Catálogo de roles del sistema (admin, driver, etc.)
`users`	`id`	Usuarios del sistema con credenciales y rol asignado
`trucks`	`id`	Registro de camiones con odómetro y estado
`truck_driver`	`id`	Historial de asignaciones camión–conductor
`mileage_logs`	`id`	Registro de kilometrajes ingresados por conductores
`maintenances`	`id`	Registros de mantenimiento (programado/en curso/completado)
`maintenance_logs`	`id`	Notas y bitácora de cada mantenimiento
`notifications`	`id`	Notificaciones generadas automáticamente por el sistema

2.2.2 Diagrama Entidad-Relación Simplificado



2.2.3 Aspectos Técnicos Destacados del Esquema

Estado del Camión (ENUM)

El campo `status` de la tabla `trucks` utiliza un tipo ENUM con tres valores posibles: `'disponible'`, `'en uso'` y `'en mantenimiento'`. Este estado se actualiza automáticamente por la lógica transaccional del sistema, garantizando consistencia sin depender de actualizaciones manuales.

Limpieza Automática de Notificaciones

El esquema incluye un **MySQL Event Scheduler** que elimina automáticamente las notificaciones con más de 6 meses de antigüedad, manteniendo la base de datos optimizada sin intervención manual.

2.3 Módulo de Autenticación y Sesiones

El sistema implementa un mecanismo de autenticación por **JWT + Cookies HTTPOnly**, proporcionando seguridad frente a ataques XSS. El flujo de autenticación es el siguiente:

1. El usuario envía sus credenciales (email y contraseña) al endpoint `POST /api/sessions/login`.
2. El sistema verifica la existencia del email en la base de datos y valida la contraseña usando `bcrypt.compare()`.
3. Si las credenciales son correctas, se genera un **JWT** firmado con la clave secreta

del servidor, con una expiración de 1 hora.

4. El token se almacena en una **cookie HTTPOnly** en el cliente, inasequible desde JavaScript del navegador.

5. Por razones de seguridad, tanto el email inexistente como la contraseña incorrecta retornan el mismo mensaje de error genérico, evitando la enumeración de usuarios.

sessionController.js - Lógica de Login

```
const login = async (req, res) => {
  const { email, password } = req.body
  const user = await usersService.getUserWithRoleByUserEmail(email)

  // Mensaje generico para no revelar si el email existe
  if (!user) return res.status(404).send({
    error: "No se ha encontrado la combinacion de usuario y contrasena."
  })

  const isValid = await passwordValidation(user, password)
  if (!isValid) return res.status(400).send({
    error: "No se ha encontrado la combinacion de usuario y contrasena."
  })

  const token = jwt.sign(userDTO, process.env.JWT_SECRET, { expiresIn: '1h' })
  res.cookie('access_token', token, { httpOnly: true, sameSite: 'none' })
    .send({ status: "success", payload: userDTO })
}
```

2.4 RF-03—Notificación de Mantenimiento

RF-03: Notificar el Mantenimiento Necesario

Autor: Hans Parra **Versión:** 1.0 — 07/10/2024

El sistema debe generar una alerta automática cuando un camión alcance el umbral de kilometraje configurado (por defecto **5.000 km recorridos desde el último mantenimiento**), notificando a los roles de administración y mantenimiento.

2.4.1 Funcionamiento del Módulo

El proceso de registro de kilometraje constituye el corazón operativo del sistema. Cuando un conductor reporta el odómetro actual de su camión, la siguiente secuencia transaccional se ejecuta:

1. **Validación de asignación activa:** Se consulta si el conductor tiene un camión asignado con `active = true` en la tabla `truck_driver`.
2. **Validación de estado:** Si el camión está `en mantenimiento`, la operación es rechazada con un mensaje explicativo.
3. **Validación de coherencia:** El nuevo kilometraje debe ser mayor al `total_mileage` actual del camión.
4. **Registro en `mileage_logs`:** Se inserta el nuevo registro con la fecha de registro.
5. **Evaluación del umbral:** Se calcula la diferencia `mileage_value` − `last_maintenance_mileage` y se compara con `maintenance_threshold`.
6. **Actualización del camión:** Se actualiza `total_mileage` y, si se superó el umbral, el estado cambia a `'en mantenimiento'`.
7. **Generación de notificaciones:** Se consultan los usuarios con roles de admin (1), superadmin (2) y mantenimiento (4) y se genera un **INSERT masivo** en la tabla `notifications`.

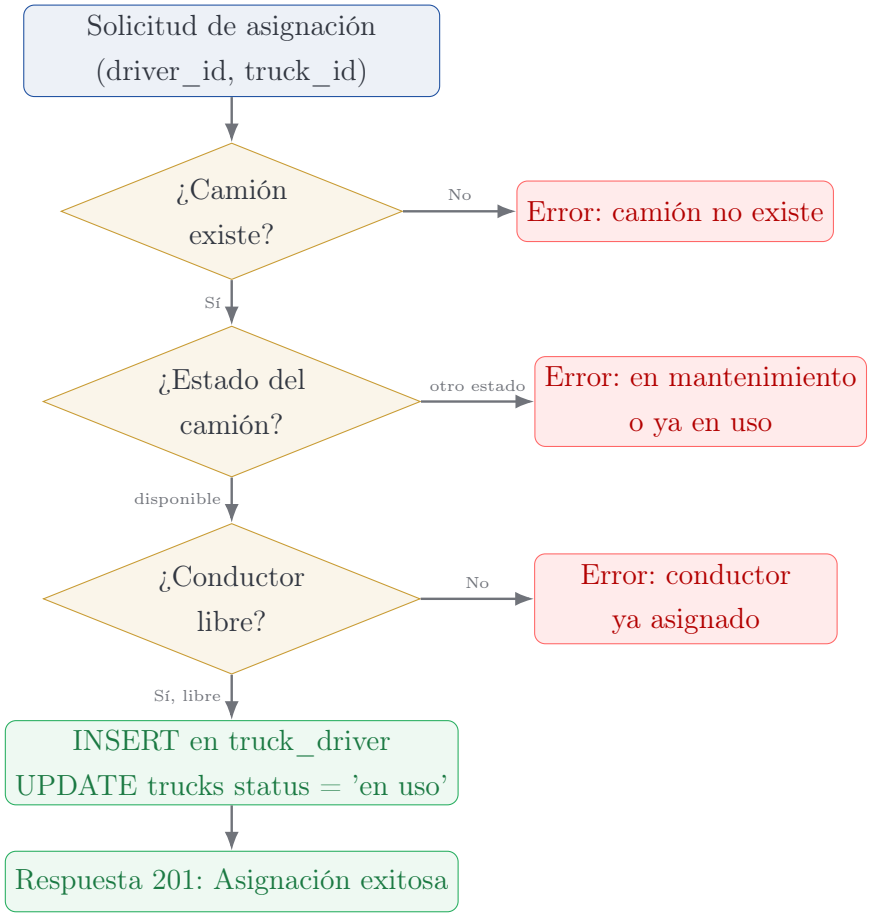
mileageDAO.js - Lógica Central de Kilometraje

```
// Logica de mantenimiento
const mileageSinceLastService = mileage_value - truck.
    last_maintenance_mileage
const needsMaintenance = mileageSinceLastService >= truck.
    maintenance_threshold

// Actualizar camion con nuevo estado
await connection.query(
  `UPDATE trucks SET total_mileage = ?, status = ? WHERE id = ?`,
  [mileage_value, needsMaintenance ? 'en mantenimiento' : '
      disponible', truck.truck_id]
)

// INSERT masivo de notificaciones a todos los usuarios con rol
    admin/maint
if (needsMaintenance) {
  const [recipients] = await connection.query(
    `SELECT id FROM users WHERE role_id IN (1, 2, 4) AND active =
        true`
```

```
  )
  const notificationValues = recipients.map(user => [
    user.id, title, message, 'mantenimiento', truck.truck_id, 'truck
      '
  ])
  await connection.query(
    `INSERT INTO notifications (user_id, title, message, type,
      reference_id, reference_type) VALUES ?`,
    [notificationValues]
  )
}
```

2.4.2 Flujo de Asignación de Camiones

El módulo de asignación implementa **transacciones SQL con bloqueo de fila** (`FOR UPDATE`) para evitar condiciones de carrera en entornos concurrentes:



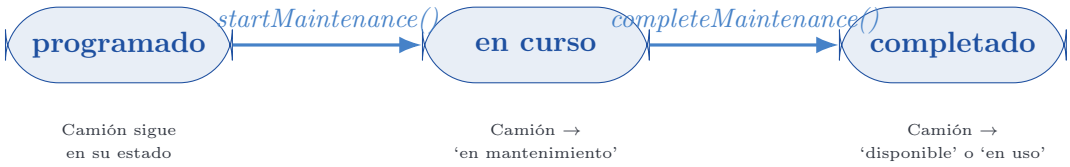
2.5 RF-04 — Gestión de Registros de Mantenimiento

RF-04: Gestión de Registros de Mantenimiento

Autor: Hans Parra **Versión:** 1.1 — 21/10/2024

El sistema permite al administrador gestionar el ciclo de vida completo de los mantenimientos (crear, consultar, actualizar y eliminar), con seguimiento de estados y sincronización automática del estado del camión asociado.

2.5.1 Ciclo de Vida de un Mantenimiento



2.5.2 Lógica de Transición de Estado del Camión

Un aspecto crítico del módulo es que al **completar** un mantenimiento, el sistema no asume ciegamente que el camión queda `disponible`. En cambio, consulta si existe una asignación activa para ese camión:

maintenanceDAO.js - Finalización Inteligente

```
// Consultar si hay un conductor asignado activo
const [activeAssignment] = await connection.query(
  `SELECT id FROM truck_driver WHERE truck_id = ? AND active = true
     LIMIT 1`,
  [truck_id]
)

// Si tenia conductor asignado, vuelve a 'en uso'; si no, pasa a '
    disponible'
const nextStatus = activeAssignment.length > 0 ? 'en uso' : '
    disponible'

await connection.query(
  `UPDATE trucks SET last_maintenance_mileage = ?, status = ? WHERE
     id = ?`,
  [completion_mileage, nextStatus, truck_id]
)
```

2.5.3 Operaciones CRUD del Módulo

Operación	Endpoint	Descripción
Crear	`POST /maintenance`	Programa un nuevo mantenimiento con fecha y tipo
Consultar todos	`GET /maintenance`	Lista todos los mantenimientos con datos del camión
Consultar por camión	`GET /maintenance/:truck_id`	Historial de mantenimientos de un camión
Iniciar	`PUT /maintenance/start`	Cambia estado a `en curso`
Completar	`PUT /maintenance/:id/complete`	Marca como completado y restaura camión
Actualizar	`PUT /maintenance/:id`	Edita campos de un mantenimiento existente
Eliminar	`DELETE /maintenance/:id`	Elimina el registro (con confirmación admin)

2.6 RF-05 — Pruebas Unitarias Automatizadas

RF-05: Pruebas Unitarias

Autor: Hans Parra **Versión:** 1.0 — 15/04/2026

El sistema debe contar con un conjunto de pruebas unitarias automatizadas que validen el correcto funcionamiento de cada módulo de forma aislada, garantizando la estabilidad del software antes de cada entrega.

2.6.1 Stack de Pruebas

El proyecto utiliza **Jest** como motor de pruebas y **Supertest** para realizar peticiones HTTP simuladas directamente sobre la aplicación Express, sin necesidad de un servidor activo en un puerto.

2.6.2 Suites de Pruebas Definidas

Archivo	Módulo Probado	Casos de Prueba Principales
`auth.test.js`	Middleware de autenticación	Protección de rutas por rol, rechazo sin token
`sessions.test.js`	Login / Logout	Credenciales correctas, inválidas, campos vacíos
`mileage.test.js`	Registro de kilometraje	Validación de umbral, rechazo de km menor o igual
`trucks.test.js`	CRUD de camiones	Asignación, reasignación, estados inválidos

2.6.3 Flujo de Ejecución

1. El desarrollador ejecuta `npm test` en la carpeta `server/`.
2. Jest carga la configuración y descubre los archivos `*.test.js` en `server/tests/`.
3. Cada suite se ejecuta de forma aislada con mecanismos de *mocking* para la base de datos.
4. El sistema valida los códigos HTTP de respuesta (200, 201, 400, 401, 403, 404, 500) y la estructura de los payloads JSON.
5. Al finalizar, se genera un reporte con el resumen de pruebas pasadas y fallidas.

Regla de Negocio

Toda funcionalidad crítica (registro de kilometraje, autenticación, gestión de mantenimiento) debe tener **al menos una prueba unitaria asociada**, y las pruebas deben ejecutarse sin errores antes de cada entrega o despliegue del sistema.

2.7 Sistema de Notificaciones

El módulo de notificaciones opera de forma completamente **pasiva**: no es invocado directamente por el usuario, sino que es disparado por los módulos de kilometraje y mantenimiento cuando se detectan condiciones relevantes.

- **Tipos disponibles:** `mantenimiento`, `asignacion`, `sistema`, `error`.
- **Referencia contextual:** Cada notificación puede incluir un `reference_id` y `reference_type` para vincularla con el recurso que la originó (ej: un camión específico).
- **Destinatarios:** Se seleccionan dinámicamente consultando los roles de admin, superadmin y mantenimiento en la tabla `users`.

- **Gestión de lectura:** Los endpoints `GET /notifications`, `PUT /notifications/:nid/read` y `PUT /notifications/read-all` permiten al usuario visualizar y marcar notificaciones.
- **Límite de consulta:** Se retornan las últimas 50 notificaciones ordenadas por fecha descendente.

3. Conclusión

3.1 Resumen de Logros

La primera etapa del proyecto **Sistema de Gestión de Flota — Transportes Hirata** culmina con la implementación completa de la capa backend, el diseño relacional de la base de datos y la especificación formal de los requerimientos funcionales RF-03, RF-04 y RF-05.

Hitos Completados en Etapa 1

- ✓ Diseño e implementación del esquema de base de datos relacional (`hirata_db`).
- ✓ Módulo de autenticación seguro con JWT + cookies HTTPOnly y hashing Bcrypt.
- ✓ API REST con arquitectura por capas (Controller / Service / DAO).
- ✓ Módulo de asignación y reasignación con transacciones ACID y bloqueo de filas.
- ✓ Módulo de kilometraje con detección automática de umbral de mantenimiento.
- ✓ Módulo de gestión de mantenimiento con ciclo de vida completo de estados.
- ✓ Sistema de notificaciones automáticas con INSERT masivo a destinatarios por rol.
- ✓ Especificación de pruebas unitarias (RF-05) con Jest y Supertest.
- ✓ Documentación técnica (README, requerimientos formales RF-03 a RF-05).

3.2 Aspectos Técnicos Destacados

El diseño muestra un nivel de madurez técnica notable para el contexto académico. En particular, se destacan los siguientes aspectos:

Consistencia transaccional: Todos los módulos críticos (asignación, kilometraje, mantenimiento) utilizan `BEGIN TRANSACTION / COMMIT / ROLLBACK` con pool de conexiones, garantizando atomicidad y coherencia de datos incluso ante errores en mitad del proceso.

Gestión de estado sincronizada: El estado del camión (`disponible / en uso / en mantenimiento`) nunca es manipulado directamente por el usuario, sino que es mantenido automáticamente por la lógica de negocio del servidor, eliminando inconsistencias.

Seguridad: La arquitectura previene enumeración de usuarios en el login, usa cookies HTTPOnly y valida los datos de entrada antes de procesarlos en cada controlador.

3.3 Trabajo Pendiente para Etapas Siguientes

Si bien la base del sistema es sólida, se identifican las siguientes áreas de mejora para las próximas etapas:

- **Pruebas unitarias:** Implementar las suites definidas en RF-05 (`auth.test.js`, `mileage.test.js`, etc.) con cobertura de los escenarios críticos.
- **Corrección en `assignmentDAO.js`:** El método `getBy` contiene un error tipográfico (`Object.key` en lugar de `Object.keys`) que debe ser corregido.
- **Middleware de autorización por rol:** Reforzar la protección de endpoints según el rol del usuario autenticado.
- **Diferencia de kilometraje:** El campo `difference_mileage` de `mileage_logs` está definido en el esquema pero no se calcula en la inserción actual.
- **Interfaz de usuario:** Completar las vistas del frontend para todos los módulos implementados en el backend.

3.4 Reflexión Final

El proyecto sienta bases sólidas para una solución informática real, aplicando buenas prácticas de ingeniería de software como la separación de capas, el uso de transacciones para proteger la integridad de los datos, la seguridad en la autenticación y la planificación de pruebas automatizadas. La arquitectura elegida es escalable y facilita la incorporación de nuevas funcionalidades en las etapas siguientes del proyecto.

4. Documentación del Proyecto

Esta sección contiene la transcripción íntegra del archivo `README.md` del repositorio, el cual sirve como guía técnica de referencia para la instalación, configuración y arquitectura general del Sistema de Gestión de Flota.

4.1 Descripción del Proyecto

Solución informática integral diseñada para automatizar el ciclo de vida de la flota de camiones de Transportes Hirata. El sistema permite el registro de conductores, la asignación de vehículos y el control estricto del kilometraje para disparar alertas de mantenimiento preventivo cada **5,000 km**.

4.2 Arquitectura del Sistema

4.2.1 Frontend

Ubicado en la carpeta `client`, construido con tecnologías estándar para asegurar ligereza y compatibilidad:

- **Vistas (HTML/Bootstrap):** Interfaces adaptables para escritorio y móviles.
- **Lógica de Cliente (JavaScript ES6):**
  - `app.js`: Gestión global de sesiones, protección de rutas por rol y control de la barra de navegación.
  - `login.js`: Gestión de autenticación y redirección inteligente según el perfil.
  - `admflota.js`: CRUD de camiones, registro de conductores y asignación dinámica de flota.
  - `driver-mileage.js`: Formulario especializado para que los conductores reporten sus recorridos.

4.2.2 Backend

Ubicado en la carpeta `server`, encargado de la seguridad y persistencia:

- **Node.js + Express** Servidor de API REST.
- **Autenticación:** Uso de Cookies y JWT para mantener sesiones seguras.
- **Seguridad de Datos:** Encriptación de contraseñas con `Bcrypt`.

4.2.3 Base de Datos (MySQL)

El esquema `hirata_db` incluye las siguientes entidades principales:

- `drivers`: Registro de conductores.
- `trucks`: Datos técnicos y odómetro de los camiones.
- `mileage_logs`: Historial de kilómetros ingresados.
- `maintenance_history`: Registro de servicios realizados.

4.3 Instalación y Despliegue

4.3.1 Requisitos Previos

- Node.js v16 o superior.
- MySQL Server v8.0.
- Navegador web moderno (Chrome, Edge o Safari).

4.3.2 Configuración del Servidor

1. Navega a la carpeta `server`.
2. Instala las dependencias: `npm install`.
3. Configura el archivo `.env` con tus credenciales de base de datos.

Archivo de Configuración .env

```
MYSQL_HOST='127.0.0.1'
MYSQL_USER='root'
MYSQL_PASSWORD='tu_password_aqui'
MYSQL_DATABASE='hirata_db'

PORT=8000
JWT_SECRET='aqui_tu_firma_secreta_de_cifrado_para_token'
```

4. Ejecuta el script SQL `dbSchema.sql` en tu instancia de MySQL.
5. Inicia el servidor mediante el comando: `npm run dev`.

4.3.3 Acceso al Cliente

1. Abre el archivo `login.html` en tu navegador.
2. El sistema se conectará automáticamente al endpoint de la API en `http://localhost:8000`.

Sistema de Gestión de Flota — Transportes Hirata

Integración de Competencias I

Etapa 1 completada — Abril 2026